
Autonomous Motion of a Modular Snake with sliding cubes

Benoît Piranda

1 Introduction

This exercise consists in realizing an application which allows to show the displacement of a set of modules forming a simple line by successive reconfigurations. The principle is to move the global set by moving the modules from one side to the other of the set. We will use for that the robots "**Sliding Cubes**" which can carry out 2 types of displacement:

- The translation to slide on a neighbor with whom it shares a face.
- The rotation around a common edge with a neighbor.

For this exercise you will need to use message communication as well as robot movement. You can use two different functions to manage the movements of the Sliding Cubes:

1. The first one allows you to know if it exists a valid motion that allows a block to reach a specified position in one move:
`module->canMoveTo(position)` returns *true* if the module can move in `position`.
2. The second one applies a move (sliding or rotation) to move the module to the given position (if the move is possible of course).
`module->moveTo(position)` starts moving the module to `position`.

Listing 1: Example of detection of valid motion

```
1 Cell3DPosition position = module->position + Cell3DPosition(0,1,0);
2 module->canMoveTo(position);
3 if (module->canMoveTo(posLeft)) {
4     module->moveTo(posLeft);
5     return true;
6 }
```

Please consider the following configuration: "*config1.xml*" made of 3 modules aligned along the *y* axis.

```
1 <?xml version="1.0" standalone="no" ?>
2 <vs>
3     <visuals>
4         <window size="1000x900" backgroundColor="#4d4dd0" />
5         <render shadows="on" grid="on"/>
6     </visuals>
7     <goal position="1,4,0"/>
8     <world gridSize="5,10,3" >
```

```

9   <camera target="50,30,17" directionSpherical="60,10,80" angle="45"/>
10  <spotlight target="100,10,80" directionSpherical="60,45,80" angle="
    25"/>
11
12  <blockList color="128,128,128" >
13    <block position="1,1,0" orientation="2" leader="true"/>
14    <block position="1,2,0" orientation="4" />
15    <block position="1,3,0" orientation="16" />
16  </blockList>
17 </world>
18 </vs>

```

2 Create a new application

The goal of the algorithm is to make the modules move on the ground so that one of them reaches the position (on the ground) indicated by the target module. The problem is that the **Sliding Cubes** can only move in contact with a neighbor. It is therefore necessary to organize successive movements. First, create a new **SBsnake** application using the VisibleSim generator, based on **Sliding Cubes** robots, which integrates the:

- **parseUserElements** method to read the final position (be careful, this function is called once, and not for all modules). Here an example of code:

Listing 2: Read the goal position

```

1  void SBsnake::parseUserElements(TiXmlDocument *config) {
2      TiXmlNode *vs = config->FirstChild("vs");
3      if (!vs) return;
4      TiXmlNode *node = vs->FirstChild("goal");
5      if (!node) return;
6      TiXmlElement *element = node->ToElement();
7      const char *attr = element->Attribute("position");
8      if (attr) {
9          goalPosition=Simulator::extractCell3DPositionFromString(
10             attr);
11          std::cout<<"goalPosition = "<< goalPosition<<std::endl;
12      } else {
13          goalPosition.set(1,5,0);
14      }
15  }

```

Be careful, as this function is only called once for all the robots, you have to use a global `goalPosition` variable in the `.cpp` file, so that all the modules have access to this information. It must not be an attribute of the `SBsnake` class.

- **parseUserBlockElements** method to define which module is a **leader**.
- **onMotionEnd** method for managing the succession of multiple movements.
- **onGLDraw** method that draws the target position in the 3D interface. The following code can be add to your code to draw a circle in the cell placed at **goalPosition** in order to show the goal of your snake in the interface of *VisibleSim*.

Listing 3: Target drawing function

```

1 void SBsnake::onGldraw() {
2     static const float thick=0.8;
3     static const float color[4]={2.2f,0.2f,0.2f,1.0f};
4     const Cell3DPosition& gs = lattice->gridSize;
5     const Vector3D gl = lattice->gridScale;
6     glDisable(GL_TEXTURE);
7     glMaterialfv(GL_FRONT, GL_AMBIENT_AND_DIFFUSE, color);
8     glPushMatrix();
9     glNormal3f(0,0,1);
10    glScalef(gl[0],gl[1],gl[2]);
11    glTranslatef(goalPosition[0],goalPosition[1],goalPosition
        [2]-0.49);
12    glBegin(GL_QUAD_STRIP);
13    for (int i=0; i<=36; i++) {
14        double cs=0.5*cos(i*M_PI/18);
15        double ss=0.5*sin(i*M_PI/18);
16        glVertex3f(thick*cs,thick*ss,0);
17        glVertex3f(cs,ss,0);
18    }
19    glEnd();
20    glPopMatrix();
21 }

```

3 Questions

Exercise 1: Autonomous motions

Question 1: Considering that the current position of a module (in grid coordinates) is stored in the attribute `module->position`, modify the code proposed by the generator in order to make the leader module move to the position `goalPosition=(1,4,0)` (marked by a red circle in the interface on Figure 1). This configuration sets the module #1, which is placed 'at the end of the line' as the leader, then it has to move up the entire line to reach the final position.

Question 2: Then complete your programme to enable the successive movement of all the modules, allowing one of the robots to reach the position `goalPosition=(1,7,0)`.

Question 3: Now, propose a method that allows the robot to reach a position that is not aligned with the set of robots. For example `goalPosition=(3,5,0)`. In this case, first move along the y-axis to reach the same y coordinate as the goal and then move along the x-axis to make one robot reach the target location.

Question 4: Consider the following configuration file ("*config2.xml*") that adds an obstacle in the way of the snake (cf. Figure 2). The modules that make up this obstacle are marked with the attribute `obstacle=true` and are drawn in blue. Complete your code to enable moving modules to pass over these modules in order to reach the goal position.

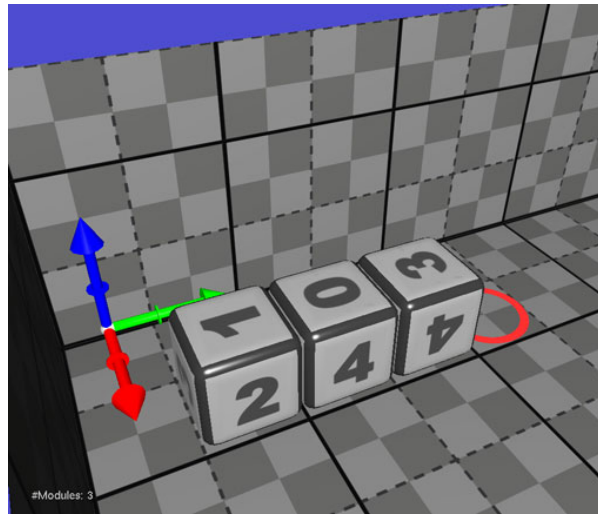


Figure 1: First configuration for motion.

```

1 <?xml version="1.0" standalone="no" ?>
2 <vs>
3   <visuals>
4     <window size="1000x900" backgroundColor="#4d4dd0" />
5     <render shadows="on" grid="on"/>
6   </visuals>
7   <goal position="1,8,0"/>
8   <world gridSize="5,10,3" >
9     <camera target="50,30,17" thetaPhiDist="60,10,80" fov="45"/>
10    <spotlight target="100,10,80" thetaPhiDist="60,45,80" fov="
11      25"/>
12    <blockList color="128,128,128" >
13      <block position="1,1,0" orientation="1" leader="true"/>
14      <block position="1,2,0" orientation="4" />
15      <block position="1,3,0" orientation="16" />
16      <block position="1,4,0" orientation="14" />
17      <block position="0,6,0" orientation="3" color="blue"
18        obstacle="true" />
19      <block position="1,6,0" orientation="5" color="blue"
20        obstacle="true" />
21      <block position="2,6,0" orientation="1" color="blue"
22        obstacle="true" />
23    </blockList>
24  </world>
25 </vs>

```

Question 5: Finally, propose a new configuration that combines a longer snake and obstacle with displacement in the y and x directions.

Upload a ZIP file containing all your project files (sources and XML files) to Moodle. You can also add pictures or videos of your simulation to your document.

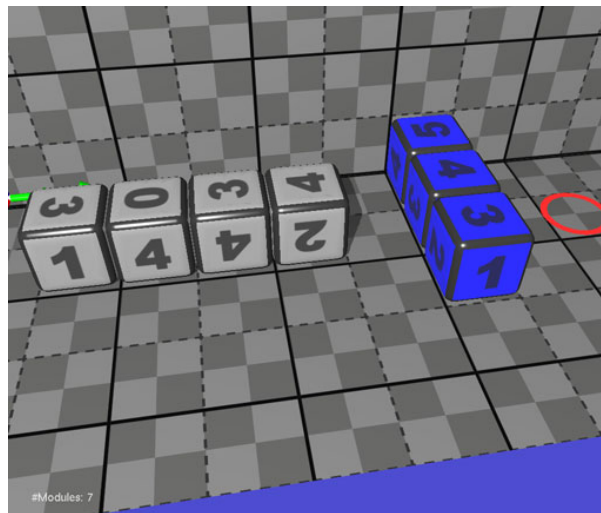


Figure 2: Configuration with obstacle.